St Francis Institute of Technology
(Autonomous Institute)
Department of Artificial Intelligence and Machine Learning
Second Year AIML Engineering (SEM-IV A.Y. 2025-26)
Web Programming Lab. Experiment Report

Experiment 9: React Side Effects & API Calls using useEffect

---

1. AIM

To implement side effects in React using the useEffect Hook and fetch data from a public API using Fetch API and async/await, and dynamically display the data as a list/cards..

---

2. LAB OBJECTIVE

- To understand React lifecycle concepts in functional components
- To learn the purpose of the useEffect Hook
- To perform API calls using Fetch API
- To use async/await for asynchronous operations
- To dynamically render fetched data in list/card format
- To handle side effects properly in React applications

---

3. LAB OUTCOME

- Use useEffect to handle side effects
- Fetch data from a public API
- Implement async/await in React
- Store API response in state
- Dynamically render lists/cards from API data
- Understand component mounting behavior

---

4. PREREQUISITE

- Functional components
- useState Hook
- Controlled components
- JavaScript promises
- Basic understanding of APIs
- JSON data format

---

5. THEORY

What are Side Effects in React?

A side effect is any operation that:

- Fetches data from an API
- Updates the DOM manually
- Sets up subscriptions or timers
- Performs logging

Side effects are operations that happen **outside normal rendering**.

5.2 React Lifecycle (Functional Component View)

In class components, lifecycle methods include:

- componentDidMount()
- componentDidUpdate()
- componentWillUnmount()

In functional components, these are replaced using: useEffect Hook

---

5.3 useEffect Hook

The useEffect Hook is used to:

- Perform side effects
- Fetch data
- Execute code after rendering

Basic syntax:

```
useEffect(() => {
  // side effect code
}, []);
```

Dependency Array:

- [] → Runs only once (on mount)
- [variable] → Runs when variable changes
- No dependency array → Runs after every render

5.4 Fetch API

The Fetch API is used to make HTTP requests.

Steps:

1. Send request to API endpoint
2. Receive response
3. Convert response to JSON
4. Store data in state

5.5 Async/Await

Async/await makes asynchronous code easier to read.

- async makes a function asynchronous.
- await waits for promise to resolve.

Benefits:

- Cleaner syntax
- Better error handling
- Avoids callback chaining

5.6 Dynamic Rendering

After data is fetched:

- Store data in state
- Use map() to iterate over data
- Render UI dynamically

---

6. PROCEDURE

Step 1: Create React Application

1. Create a new React app.
2. Navigate to project directory.
3. Start development server.
4. Remove unnecessary default files.

Step 2: Choose a Public API

Use a free public API such as:

- JSONPlaceholder (posts/users)
- Dummy products API
- Public REST API

Example API:

https://jsonplaceholder.typicode.com/users

Step 3: Create a Functional Component

1. Create a component named UserList (or similar).
2. Import and render it inside App.js.

Step 4: Initialize State

Inside the component:

1. Create state to store:
    - API data (array)
    - Loading status
    - Error message (optional)

Initial values:

- Data → empty array
- Loading → true
- Error → null

Step 5: Implement useEffect Hook

1. Import useEffect.
2. Inside useEffect:
    - Create an async function.
    - Call the API using Fetch.
    - Await the response.
    - Convert response to JSON.
    - Update state with received data.
    - Set loading to false.
3. Call the async function inside useEffect.
4. Use empty dependency array to run only once.

Step 6: Handle Loading State

Before displaying data:

- If loading is true → Show “Loading...” message.

Step 7: Handle Errors

Use try-catch:

- If error occurs → Store error message in state.
- Display error message in UI.

Step 8: Dynamically Display Data

1. Use map() to iterate over data array.
2. Render each item as:
    - Card format (recommended)
      OR
    - List format
3. Display relevant fields such as:
    - Name
    - Email
    - Company
    - Phone

Step 9: Style Cards (Optional Enhancement)

- Add simple CSS styling
- Use flexbox/grid layout
- Add borders/shadows

Step 10: Test the Application

- Data loads automatically on page load
- No page refresh required
- Cards are generated dynamically
- Errors are handled properly
- No infinite re-render occurs

---

7. RESULTS / OUTCOME EXPECTED

- How is data fetched from the public API in this experiment? Explain the process.

- How does the useEffect Hook ensure that the API call runs only once when the component loads?
- Why is async/await used while performing API calls in React? How does it simplify asynchronous operations?
- Why must the fetched API data be stored in state? What happens after storing it?
- How are the fetched data items displayed dynamically as a list or cards in the application?
- How are loading and error conditions handled during the API request?

---

8. CONCLUSION

- What is a side effect in React?
- Why is useEffect used for API calls?
- What happens if dependency array is removed?
- How does async/await simplify API calls?
- How is dynamic rendering achieved?

---

9. POST-EXPERIMENT QUESTIONS

- What is the difference between useEffect and lifecycle methods?
- What causes infinite re-rendering in useEffect?
- Why must API data be stored in state?
- What is the purpose of dependency array?
- How can we clean up side effects?

---

10. REFERENCES

- React Official Documentation – https://react.dev
- MDN Web Docs – Fetch API
- JSONPlaceholder API Documentation
- W3Schools React Tutorial